

There are three perspectives or uses of logic that we will consider.

One is the correspondence of logic with data, at least categorical data, epitomized by entries with two possible values. Formal logical expressions then correspond to formal expressions about data.

Another perspective is logic for the use of arguments and reason. In the extreme case of being rigorous, there is mathematical proofs in the language of formal logic. A valid proof falls into one of a few categories. One category is proof by contradiction where one assumes the statement is false and derives a contradiction.

In order to verify a proof, there are rules of inference. If a proof follows allowed rules of inference, then it is a rigorous proof. Otherwise, the proof is iffy, and if we allow iffy proofs, false statements could be accepted as true. Ideally, rules of inference would be correct and powerful, enabling the proofs of true statements. So the middle perspective of logic is the study of rules of inference.

Logic

True and False

These are called boolean values true or false, 1 or 0.

A common implementation, and a common way to think about it, is that true = 1 and false = 0

For one thing, any list of 0,1's can be thought of as a list of false, trues. And since any list can be thought to correspond to variables, e.g., $x[1] = x_1$, and list of boolean values as an assignment of variables to boolean values.

Since any number can be written in base 2 with 0,1's that also means any number corresponds to a list of boolean values.

Working our way through the possible correspondences to list of boolean values, there is an underlying point of view that any discrete object can be represented with logical variables. This extends to any categorical data.

The abstract status of real numbers is less clear, but as long as the numbers in data come with a finite number of digits (or other finite representation) then they too can be represented by logical variables.

To each problem we have to use the right tools. For instance, it would not be productive to start using logic and formal manipulations to add two numbers, or write $x < y$.

Usually logic is useful when the output is true or false, or if one needs to rigorous proof.

Operators

And

x and y is true when both x and y are true, otherwise false

$x \&\& y$ (in Python it is “x and y”)

$x \& y$ for bitwise operations

$x y \bmod 2$ = ordinary multiplication mod 2

$x \wedge y$

Or

x or y is true when either x and y are true, or both, otherwise false

$x || y$ (in Python it is “x or y”)

$x | y$ for bitwise operations

$x \vee y$

Not

not x is true when x is false, and not x is false when x is true

$!x$

$\sim x$ for bitwise operations (depends on what an integer is)

$\neg x$

Xor

x xor y is true when either x and y are true, but not both both, otherwise false

usually not implemented as a builtin logical operator

$x \wedge y$ for bitwise operations

$x + y \bmod 2$ = ordinary addition mod 2

$x \vee y$

Implies

x implies y is true when either x and y are true, or x is false, otherwise false when x is true and y is false

usually not implemented as a builtin logical operator

$x \Rightarrow y$

Equivalent

x equivalent y is true when either x and y are the same, otherwise false

$x == y$

$x \Leftrightarrow y$

Nand

x nand y is false when x and y are true, otherwise true

$\text{not}(x \text{ and } y)$

usually not implemented as a builtin logical operator

$x \bar{\wedge} y$

Nor

x nor y is true when either x and y are false, otherwise false

$\text{not}(x \text{ or } y)$

usually not implemented as a builtin logical operator

$x \bar{\vee} y$

Formal Logic

Traditionally logic might be considered to be useful for statements in the form of English sentences (or in another natural language).

Formal logic is about symbols and operators, which sometimes correspond to statements that could be in the form of English sentences.

Through formalization one can focus on allowed rules of inference and also abstract features of the collection of logical expressions.

Logical Expressions

Propositional Logic

The logical expressions correspond to boolean functions.

If all the variables have a value then the operators in the expression can be evaluated from the bottom up to provide a truth value for the expression.

For example, if $x = \text{True}$ and $y = \text{False}$ and $z = \text{False}$

Predicate Logic

The propositional logic cannot allow any explicit mention of for all, denoted by $\forall$, or there exists, denoted by $\exists$

$\forall \epsilon > 0, \exists \delta > 0$ such that $|x| < \delta$ implies that $|\sin(x)| < \epsilon$

For certain database queries there is almost a direct relationship with predicate logic.

The database corresponds to the universe of possibilities.

One can ask if there is a car in the database that is blue:

$\exists$ blue car

Or one can assert that every car has an owner.

$\forall$ cars $\exists$ owner of the car

In order to go beyond propositional logic, one needs more than one predicate in a statement.

Sets and Functions

Traditionally in mathematics, one is allowed to express more than what can be expressed in predicate logic.

For any function $f(\text{car})$ from cars to people, there is a function $g(\text{person})$ from people to people such that $g(f(\text{car})) = \text{owner of the car}$.

For any map $x(n)$ from the natural numbers to the closed interval $[0,1]$, there is an increasing map $f(k)$ from the natural numbers to itself such that the composition $x(f(k))$ is a converging sequence.

Any subset X of the real line has a topological closure $\overline{X}$ which is another subset such that

Truth Tables

In general, two expressions are the same if they have the same truth table.

Operators Truth Tables

Operators can also be defined by their truth tables.

Out[]:=

and	x	y	$x \wedge y$
	True	True	True
	True	False	False
	False	True	False
	False	False	False
or	x	y	$x \vee y$
	True	True	True
	True	False	True
	False	True	True
	False	False	False
xor	x	y	$x \vee y$
	True	True	False
	True	False	True
	False	True	True
	False	False	False
implies	x	y	$x \Rightarrow y$
	True	True	True
	True	False	False
	False	True	True
	False	False	True
equivalent	x	y	$x \Leftrightarrow y$
	True	True	True
	True	False	False
	False	True	False
	False	False	True
nand	x	y	$x \bar{\wedge} y$
	True	True	False
	True	False	True
	False	True	True
	False	False	True
nor	x	y	$x \bar{\vee} y$
	True	True	False
	True	False	False
	False	True	False
	False	False	True

Disjunctive Normal Form

If there are n variables then there are 2^n rows in a truth table.

There are $2^{(2^n)}$ possible boolean functions.

Can they all be realized as logical expressions?

It depends on what operators are allowed.

With Or, And, Not, it is possible using disjunctive normal form, which only has one Or if you allow the multi-argument Or.

Out[]:=

x	y	z	expression?
True	True	True	False
True	True	False	False
True	False	True	False
True	False	False	True
False	True	True	True
False	True	False	True
False	False	True	True
False	False	False	False

With the disjunctive normal form, one highlights the cases where it is true.

Out[]:=

x	y	z	expression?
True	True	True	False
True	True	False	False
True	False	True	False
True	False	False	True
False	True	True	True
False	True	False	True
False	False	True	True
False	False	False	False

There are four rows, so there are four ways this expression could be true. It is true if and only if one of them is the case.

For each of these rows, if variable x_i is true, no modification is needed, but if x_i is false, then $\neg x_i$ is true.

$$x \wedge \neg y \wedge \neg z$$

$$\neg x \wedge y \wedge z$$

$$\neg x \wedge y \wedge \neg z$$

$$\neg x \wedge \neg y \wedge z$$

The expression is true if and only one of these is true, so we can use Or to combine them.

$$(x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z)$$

The minimal disjunctive normal form eliminates variables when they don't matter. For instance here, we can use

$$(x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z)$$

$$(x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y) \vee (\neg x \wedge \neg y \wedge z)$$

There is one more variable that can be eliminated.

$$(x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y) \vee (\neg x \wedge z)$$

The disjunctive normal form is And+Not-expressions connected by Or.

```
BooleanConvert[Equivalent[x, y], "DNF"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$(x \wedge y) \vee (\neg x \wedge \neg y)$$

In machine learning, when one is looking for a categorical type of output, like is a picture nighttime or daytime, one is also looking for a function. It usually is the case that the function is not correct on all inputs, but this is worth producing a function which is much simpler than a large complicated expression.

Equivalent Expressions

Two expressions are equivalent if they evaluate to the same boolean value for every possible value of the variables.

Conjunctive Normal Form

The conjunctive normal form is Or+Not-expressions connected by And.

```
BooleanConvert[Equivalent[x, y], "CNF"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$(\neg x \vee y) \wedge (x \vee \neg y)$$

Algebraic Normal Form

```
BooleanConvert[Equivalent[x, y], "ANF"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg (x \underline{\vee} y)$$

Boolean algebra can be expressed with multiplication being And and addition being Xor. Not is 1-x.

```
BooleanConvert[Equivalent[x, y], "ANF"] /. {Xor -> Plus, And -> Times, Not -> (1 - # &) }
```

```
Out[ ]= 1 - x - y
```

Nand

Everything can be expressed using one operator, e.g., Nand is complete.

```
Simplify[Nand[x, x]] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg x$$

Once we get not, then And is easy

```
Simplify[! Nand[x, y]] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$x \wedge y$$

and Or is only a little more complicated.

```
Simplify[Nand[! x, ! y]] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$x \vee y$$

Converting something simple into Nand can be complicated.

```
BooleanConvert[Equivalent[x, y], "NAND"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$(x \bar{\wedge} y) \bar{\wedge} (\neg x \bar{\wedge} \neg y)$$

```
BooleanConvert[Or[z, Equivalent[x, y]], "NAND"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$(x \bar{\wedge} y) \bar{\wedge} (\neg x \bar{\wedge} \neg y) \bar{\wedge} \neg z$$

Others

Implies alone is not complete, but implies and not is.

```
BooleanConvert[And[x, y], "Implies"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg (x \Rightarrow \neg y)$$

```
BooleanConvert[Or[x, y], "Implies"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg x \Rightarrow y$$

Since Not is available, we don't need to produce it.

```
BooleanConvert[Not[x], "Implies"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg x$$

It can be fairly complicated.

```
BooleanConvert[Or[z, Equivalent[x, y]], "Implies"] // TraditionalForm
```

```
Out[ ]//TraditionalForm=
```

$$\neg z \Rightarrow ((x \Rightarrow \neg y) \Rightarrow \neg (\neg x \Rightarrow y))$$

in words z or (x=y) is the same as

not(z) implies that the implication x implies not(y) itself implies that the implication not(x) implies y is false.

Random Expressions

These are just a sample of expressions equivalent to $x \vee (y \wedge z)$


```

(x ∨ y) ∧ (¬ x ∧ (z ∧ y))
(y ∨ x) ∧ (¬ x ⇒ z)
((y ∧ z) ∨ y ⇒ y ∧ ¬ x) ⇒ (y ∧ z) ∨ x
y ∧ z ⇒ x
z ∧ y ⇒ x
((y ⇒ x) ∨ (y ⇒ z)) ∧ z ⇒ x
¬ x ⇒ z ∧ y
¬ (z ∧ y) ⇒ x ∨ x
(x ∧ x) ∧ (y ∧ z ⇒ x)
(x ∧ x) ∧ ¬ (z ∧ (z ⇒ y))
¬ x ∧ (z ∧ y)
¬ (¬ x ∧ (y ∨ (y ⇒ z)))
Out[4]= ¬ ((¬ x ∧ z ∧ (y ∨ y)) ∨ ¬ x)
x ∨ (y ∧ z)
x ∨ (y ∧ (y ⇒ z))
x ∨ (z ∧ y)
x ∨ (¬ (y ∧ z) ∧ (z ∨ x))
x ∨ (y ∧ y ∧ ¬ x ∧ z)
x ∨ ¬ (z ∧ y)
(x ∧ (x ∨ z)) ∨ ((x ∧ z) ∧ ((z ∨ z) ∧ (x ∨ y)))
(z ∧ y) ∨ x
(z ∧ y) ∨ (x ∧ x)
(z ∧ x ⇒ x) ∨ ((y ∧ z) ∧ (x ∨ (x ⇒ y)))
(((z ⇒ y ∧ y) ⇒ (x ∨ y ⇒ z)) ∧ x) ∨ (z ∧ y) ∨ x
(y ∧ z) ∨ x ∨ (x ∧ z) ∨ (y ∧ x)

```

Since we can keep building larger and larger expressions, the number of formal expressions is infinite. There are only 256 boolean functions on 3 variables. So it is not surprising that each function can be represented an infinite number of ways.

Consistency

False

If there are no cases where the expression is true, it is equivalent to the statement False.

If we have a sequence of statements joined by And which is equivalent to false, then we say those statements are inconsistent.

$x \wedge \neg x$

x and $\neg(x)$ is obviously inconsistent.

Propositional logic, where it is just variables and logical operators, is relatively straightforward. All true statements can be proven by rules of inference.

The situation for predicate logic, when there are quantifiers like $\forall$ or $\exists$, and the situation for set theory is more complicated.

In any case, rules for inference will let us construct valid proofs.

Inconsistency will also be useful for deriving proofs. One type of proof, the proof by contradiction,

requires the derivation of an inconsistency.

Rules of Inference

Rules of inference transform a logical expression to an equivalent logical expression.

Often the goal is to simplify a logical expression as part of a proof, but it could also happen that the rule of inference makes things more complicated. In a proof, it may be necessary to make equivalent expressions that are more complicated so they can be simplified later.

There are rules of inference for each type of logic: propositional and predicate.

Among the possible equivalences, certain ones have been chosen for textbooks. They typically only involve the standard three operators: And, Or, Not

De Morgan's Rules

Out[#]=

$(\neg (p \wedge q)) = (\neg p \vee \neg q)$
$(\neg (p \vee q)) = (\neg p \wedge \neg q)$

for instance, not(p and q) is the same as (not p) or (not q)

Formally there is a symmetry between And and Or which comes from switching the roles of True and False.

Other Textbook Rules

Out[#]=

$p = (p \wedge p)$	idempotent of and
$p = (p \vee p)$	idempotent of or
$(p \wedge q) = (q \wedge p)$	commutativity of and
$(p \vee q) = (q \vee p)$	commutativity of or
$p = (\neg (\neg p))$	law of double negation
$p = (p \wedge (p \vee q))$	adsorption
$p = (p \vee (p \wedge q))$	adsorption
$(p \wedge \neg p) = (q \wedge \neg q)$	law of noncontradiction
$(p \vee \neg p) = (q \vee \neg q)$	law of excluded middle
$(p \wedge (q \wedge r)) = ((p \wedge q) \wedge r)$	associativity of and
$(p \vee (q \vee r)) = ((p \vee q) \vee r)$	associativity of or
$(p \wedge (q \vee r)) = ((p \wedge q) \vee (p \wedge r))$	first distributive law
$(p \vee (q \wedge r)) = ((p \vee q) \wedge (p \vee r))$	second distributive law

Which of these are needed for an axiom system which forces the operators to be logic? There are many equivalent choices of axiom systems. Here is a standard list.

Out[]=

$(p \wedge q) = (q \wedge p)$
$(p \vee q) = (q \vee p)$
$p = (p \wedge (q \vee \neg q))$
$p = (p \vee (q \wedge \neg q))$
$(p \wedge (q \vee r)) = ((p \wedge q) \vee (p \wedge r))$
$(p \vee (q \wedge r)) = ((p \vee q) \wedge (p \vee r))$

Rules for Predicate Logic

One restriction for expressions is that the variables must be scoped.

That means, $\forall x P(x)$ can only have x inside the $P(x)$. It can't be sitting outside.

The basic axioms (Tarski) for predicate logic are (complicated)

Some basic rules for predicate logic involve negation

Out[]=

$(\forall p \neg q) = (\neg \exists p q)$
$(\exists p \neg q) = (\neg \forall p q)$

These form the basis for the proof by counterexample.

There are also two rules that correspond to the usual interpretation of $\forall$ and $\exists$:

Existential Generalization

If one knows that Delenn's car is blue then one can infer that there exists a blue car.

More abstractly, if one establishes

$P(a)$ as true

then it follows that

$\exists x, P(x)$

This is called existential generalization.

Combined with the laws for negating quantifiers above, we get

Universal Instantiation

If one establishes that all car's are not red (which could be the case in a dataset) then one can infer that Delenn's car is not red.

More abstractly, if one establishes

$\forall x, P(x)$ as true

then it follows that

$P(a)$

where a is an instance of x .

This is called universal instantiation.

Proof

We don't prove things by finding an example, except for the proof by counterexample.

Otherwise examples are not sufficient for a proof. They can count as evidence, and they can provide insights. So examples have their role to play.

Excluded middle

We will assume that a statement is either true or false. There is no middle. This is called the excluded middle.

So to prove a statement that is true, and otherwise, we might be called upon to disprove a statement.

Disproving P means proving not P.

Allowed steps

The steps need to follow accepted rules of inference.

The structure of a proof usually follows one of the following forms, and depending on the form some standard steps don't need explicit mention.

Direct proofs

A direct proof starts with the givens and proceeds by a series of steps.

Each step is a consequence of the givens and the previous steps, which themselves are consequences of the givens, so each step ultimately is a consequence of the givens.

The transformation from one step to the next is allowed by the rules of inference.

Here is a simple example.

Recall that these are the standard axioms of logic. We give them labels so we can refer to them to justify our steps.

$(p \wedge q) = (q \wedge p)$	(1)
$(p \vee q) = (q \vee p)$	(2)
$p = (p \wedge (q \vee \neg q))$	(3)
$p = (p \vee (q \wedge \neg q))$	(4)
$(p \wedge (q \vee r)) = ((p \wedge q) \vee (p \wedge r))$	(5)
$(p \vee (q \wedge r)) = ((p \vee q) \wedge (p \vee r))$	(6)

Out[]=

That means that, without interpreting the symbols as And, Or, Not, we should be able to derive the law of noncontradiction:

$$p \wedge \neg p = q \wedge \neg q$$

We could consider the axioms as givens, and the rules of inference are simple operator substitutions.

Let the p in formula (4) be the expression $p \wedge \neg p$,
then we get

$$p \wedge \neg p = (p \wedge \neg p) \vee (q \wedge \neg q)$$

Next use commutativity of $\vee$ in formula (2) to get

$$(p \wedge \neg p) \vee (q \wedge \neg q) = (q \wedge \neg q) \vee (p \wedge \neg p)$$

Noting the symmetry we again reuse formula (4) except now with $(q \wedge \neg q)$ in the place of p and p in the place of q :

$$q \wedge \neg q = (q \wedge \neg q) \vee (p \wedge \neg p)$$

Following the chain of equalities we derived $p \wedge \neg p = q \wedge \neg q$

Note that no matter what example we use $p \wedge \neg p$ is false, so the examples give some insight why this is true.

Do the examples help with the proof? Maybe only a little bit. Since we see that we can interchange roles of p and q , the commutative law might be involved.

Another example of a direct proof is an ordinary calculation.

How would you prove that $(1 + \sqrt{5})/2$ is a root of $x^2 - x - 1$? Plug it in and show it is zero. The rules of inference would involve the rules of algebra and the definition of $\sqrt{5}$

Direct proofs are often the best kind of proof, when there is a choice. All proofs provide a verification but a direct proof can provide insight.

Proof by Counterexample

This method is only for statements $\neg \forall x, P(x)$

So this is typically applicable when disproving $\forall x, P(x)$

An obviously false statement is that all real numbers are positive:

$$\forall x, x > 0$$

which we can disprove by providing a counterexample.

Let $x = -1$. Then it is false that $x > 0$.

We would stop there because the rest of the argument is always the same.

$$\text{not } (-1 > 0) = \text{not } P(-1)$$

$\exists x, \neg P(x)$ by existential generalization

$\neg \forall x, P(x)$ by the negation rule of inference for quantifiers.

In practice, what is required to be shown is an example, and show the statement is false for the example. Then we call this a counterexample.

Proof by Contradiction

If we can show that $\neg P$ is false, then by double negation we get that P is true.

Proof by Induction

Proof by Cases

The proof could be simpler if an extra hypothesis P is made. So first assume that P is true and use that to create a proof. Then assume P is false and use that to create a proof.

In general, the cases could depend on the value of a variable. One must be careful that all possibilities are covered by the cases.

For example, consider the statement:

the solutions to $\alpha x + \beta y = 0$ form a line if $\alpha^2 + \beta^2 = 1$

Case 1: $\beta \neq 0$

Case 2: $\beta = 0$

There would be a proof for case 1, and a proof for case 2, depending on the criterion for being a line and what facts have already been established. For instance, if it has already been established that $y = m x + b$ is a line then that could be used in case 1, but not in case 2.

Any number of cases can be used, not just two cases.

Sometimes a case is handled by showing it cannot happen because it leads to a contradiction. For instance, given three distinct real numbers a, b, c one might use trichotomy to make eight cases (because we assume they are distinct they are unequal):

Out[5]=

1	$a < b \wedge a < c \wedge c < b$
2	$a < b \wedge c < a \wedge b < c$
3	$a < b \wedge c < a \wedge c < b$
4	$b < a \wedge a < c \wedge b < c$
5	$b < a \wedge a < c \wedge c < b$
6	$b < a \wedge c < a \wedge b < c$
7	$b < a \wedge c < a \wedge c < b$
8	$a < b \wedge a < c \wedge b < c$

Two of these cases contradict the trichotomy property (or also the well-ordered property) of the reals.

Out[9]=

1	$a < b \wedge a < c \wedge c < b$	
2	$a < b \wedge c < a \wedge b < c$	contradiction
3	$a < b \wedge c < a \wedge c < b$	
4	$b < a \wedge a < c \wedge b < c$	
5	$b < a \wedge a < c \wedge c < b$	contradiction
6	$b < a \wedge c < a \wedge b < c$	
7	$b < a \wedge c < a \wedge c < b$	
8	$a < b \wedge a < c \wedge b < c$	

There is no point in breaking it up into cases if you don't use that.

Argue by Symmetry

When the possible cases correspond to a simple change of notation, one might argue by symmetry.

For instance, take the intermediate value theorem. Assume $f(x)$ is continuous on the closed interval $[a,b]$. Given a value c in between $f(a)$ and $f(b)$ there is an x in $[a,b]$ such that $f(x) = c$.

The statement that c is in between $f(a)$ and $f(b)$ is a little clunky, and to make it more formal, one might say something like this:

Assume $f(x)$ is continuous on the closed interval $[a,b]$. Further assume that $f(a) \leq c \leq f(b)$ then... By symmetry, the conclusion holds if $f(b) \leq c \leq f(a)$

Depending on the level of detail expected, one might have to argue that the $f(b) \leq c \leq f(a)$ case follows from the $f(a) \leq c \leq f(b)$ case by considering $-f(x)$.

Lemmas

A lemma is a useful intermediate result. After it has been proven, it can be used in any proof.

It might be useful because it can be used many times.

It might be useful because it helps organize a longer proof, usually by breaking it into chunks.

Corollary

A corollary is a result based on a previous lemma or theorem. Usually its proof is shorter, and it involves an application that might not be completely obvious.

Theorem

A theorem is a result, usually more serious than a lemma. Usually anything with a difficult proof, either long or complex, is called a theorem. It could also be a result that is judged as important.

Conjecture

A conjecture is not proven. Usually it is well-defined so that it is believed to be either true or false. There should at least be some evidence that it is true.

Only rarely is a theorem based on a conjecture, in which case the dependence is made clear. For instance, the Riemann Hypothesis is a famous conjecture, and there are several theorems of the form (If the Riemann Hypothesis is true) then